Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Veri sıkıştırma (data compression)
  - Amaç, verinin daha az yer kaplayarak saklanmasıdır. Örneğin 100 KB'lık bir veri 80 KB'a indirilirse %20 tasarruf sağlanır.
  - Sıkıştırma iki işlemden oluşur: kodlama (encoding) ve kod çözme (decoding). Veri bir sıkıştırma algoritmasından geçirilerek kodlanır. Kullanılacağı zaman tekrar çözülür.
  - Çözme sonunda aynı veri elde ediliyorsa kayıpsız (lossless), elde edilemiyorsa kayıplı (lossy) sıkıştırmadır.
  - Her verinin içindeki örüntü farklı olduğu için veriye uygun sıkıştırma algoritması seçilmelidir.
  - Her dosyanın sürekli sıkıştırılmamasının nedeni maliyettir: her kodlama ve çözme işlemi ek işlemci gücü harcar. Sıkıştırılmış veri doğrudan okunamaz, önce çözülmesi gerekir.
  - Kodlama/kod çözme kavramı veri güvenliğinde (data security) de kullanılır: şifreleme ve şifre çözme.
  - Sıkıştırma ile şifreleme farklıdır: sıkıştırma veriyi daha az bitle ifade etmeyi, şifreleme ise verinin karşı tarafça anlaşılmamasını amaçlar.
- Sıkıştırma yöntemleri
  - Run-Length Encoding (RLE): art arda tekrar eden değer bir kez yazılır ve tekrar sayısı eklenir (ekrandan, 00:20:50).
    - Metin örneği: `AAABBCCCDDEEF` (13 byte) → `3A2B3C2D2E1F` (12 byte). Kazanç çok düşüktür.
    - Piksel örneği: `10 10 10 10 255 255 255 0 0 0 0 5` (12 byte) → `4 10 3 255 4 0 1 5` (8 byte). 4 byte, yani yaklaşık %33 kazanç sağlanır.
    - Değerler bellekte ayrı hücrelerde tutulduğu için "3 255" ile "32 55" karışmaz.
    - Tekrarın çok olduğu verilerde (gökyüzü, deniz, çimen gibi geniş tek renkli alanlar) verimlidir. Tekrar yoksa veriyi büyütür: `ABCDEF` (6 byte) → 12 byte olur.
  - Frekansa bağlı kodlama (frequency dependent / variable-length encoding): sık kullanılan semboller daha az bitle, seyrek kullanılan semboller daha çok bitle kodlanır. Türkçede sesli harfler (a, e) sık, ğ, ş, ü gibi harfler daha seyrek kullanılır. Huffman kodlaması bu yaklaşıma örnektir ve ileriki derslerde görülecektir.
  - Göreli kodlama (relative / differential encoding): özellikle videoda kullanılır. Art arda gelen iki kare arasındaki fark bulunur ve yalnızca değişen kısım gönderilir. Değişmeyen pikseller önceki kareden alınır.
  - Sözlük kodlaması (dictionary encoding): kelimeler, sözlükteki daha küçük kelime blokları kullanılarak saklanır. Örneğin "balıkesir" kelimesi "balık" bilgisi üzerine "esir" eklenerek ifade edilebilir.
- Sıkıştırmanın faydaları
  - Daha az depolama alanı.
  - Ağdaki sınırlı bant genişliğinin daha verimli kullanılması. Büyük veriler trafikte sıkışma ve gecikmeye yol açabilir.
  - Mobil veri kotası gibi iletim maliyetlerinin azalması.
  - Ağ üzerinden gönderilen veriler günümüzde düz metin (plain text) olarak değil, kodlanarak (şifrelenerek) gönderilir.
  - Ham veri (raw data): veri hangi kaynaktan elde edildiyse, işlenmemiş hâline verilen isimdir. İçinde hatalı bilgiler olabilir ve düzeltilmesi gerekebilir.

- Sıkıştırma kullanan formatlar
  - Genel dosya arşivleme: RAR, ZIP, 7z. Hız ve sıkıştırma oranı arasında seçim imkânı sunarlar.
  - Özelleşmiş formatlar: MP3 ve FLAC (ses), JPEG ve GIF (resim), MPEG ve MKV (video).
  - Yeni formatlar en çok, boyutları ve kaliteleri sürekli artan resim ve video (multimedya) verileri için geliştirilir.
- OCR (Optical Character Recognition)
  - Resim içindeki yazıyı metne dönüştürme yöntemidir. Harfler tek tek tespit edilir. Boşluklardan kelimeler, satır boşluklarından satırlar çıkarılır.
- İletişim hataları (communication errors)
  - Bilgisayarın iki birimi arasında (bellekten CPU'ya, diskten belleğe) ya da iki bilgisayar arasında veri aktarılırken oluşan hatalardır. Kablosuz ortamda fiziksel engeller veya radyo frekansı sorunları da hataya yol açabilir.
  - En temel hata türü bitlerin değişmesidir: 0'ın 1, 1'in 0 olması. Tek bir bitin değişmesi bile verinin anlamını tamamen değiştirebilir.
  - İki yaklaşım vardır: hata tespiti (error detection) ve hata düzeltme (error correction). Tespit daha ucuzdur. Düzeltme daha maliyetlidir.
- Eşlik biti (parity bit) (ekrandan, 01:02:40)
  - Veriye eklenen fazladan bir bittir (additional bit).
  - Tek eşlik (odd parity): toplam 1 sayısı tek olacak şekilde eşlik biti seçilir. Çift eşlik (even parity): toplam 1 sayısı çift olur.
  - Örnek (tek eşlik): `01000001` verisinde iki tane 1 vardır → eşlik biti 1 eklenir → 1 `01000001` (toplam üç tane 1). `01000110` verisinde üç tane 1 vardır → eşlik biti 0 olur.
  - Alıcı 1'lerin sayısının beklenen teklik/çiftlikte olup olmadığına bakarak hatayı anlar.
  - Sınırı: yalnızca tek sayıda bit değişirse hata yakalanır. İki bit birlikte değişirse 1 sayısının teklik/çiftliği bozulmaz ve hata fark edilmez.
  - İlgili kavramlar: check byte, CRC (Cyclic Redundancy Check), checksum. Hoca bunlara genel kültür olarak bakılmasını önerir.
- Hamming uzaklığı (Hamming distance) ve hata düzeltme (ekrandan, 01:17:40)
  - İki bit deseni arasındaki farklı bit sayısıdır.
  - 8 sembol 3 bitle ifade edilebilir. Örnekte ise 6 bit kullanılmış ve kodlar, her iki sembol arasındaki Hamming uzaklığı en az 3 olacak şekilde seçilmiştir:

Sembol	Kod
A	000000
B	001111
C	010011
D	011100
E	100110
F	101001
G	110101
H	111010

  - Alınan desen `010100` hiçbir sembole eşit değildir. Her sembole uzaklığı: A=2, B=4, C=3, D=1, E=3, F=5, G=2, H=4. En yakın sembol D'dir (uzaklık 1). Desen D olarak düzeltilir.
  - Kodlar arası uzaklık en az 3 olduğu için tek bitlik bir hata her zaman tek bir en yakın sembole götürür. İki sembole eşit uzaklıkta kalma durumu oluşmaz.
  - Bedeli: 3 bit yerine 6 bit kullanılır.
  - Hata tespit edildiğinde verinin yeniden gönderilmesi istenebilir. Yeniden almak pahalıysa, veriyi biraz daha büyük taşıyıp hatayı yerinde düzeltmek tercih edilebilir.
  - Benzer bir fikir disklerde RAID düzenlerinde görülür: alanın bir kısmı, veri bozulduğunda kurtarmak için tekrar eden (redundant) bilgiye ayrılır.
- Algoritma
  - Tanım (ekrandan, 01:38:55): "An algorithm is an ordered set of unambiguous, executable steps that defines a terminating step." Yani algoritma; sıralı, belirsizlik içermeyen (unambiguous), uygulanabilir (executable) ve sonlanan adımlar kümesidir. (Ders sonrası not: kaynak kitaplarda tanım büyük olasılıkla "… that defines a terminating

process” biçimindedir.)
  - Algoritma yalnızca bilgisayara özgü değildir: yemek tarifi, alışveriş planı, zeka küpü çözümü, mobilya yapım aşamaları da birer algoritmadır.
  - Bir problemin tek bir algoritması yoktur. Aynı hedef için farklı yöntemler olabilir ve zamanla daha iyisi bulunabilir (daha hızlı, daha az yer kaplayan).
  - Belirlilik: her adımın tek bir anlamı olmalıdır. “Su dök” yerine “100 ml su dök” gibi kesin ifade gerekir. Programlama dillerinde ve algoritma gösterimlerinde her ifadenin tek bir anlamı vardır.
  - Sonlanma: her algoritmanın bir başlangıcı ve bir bitişi olmalıdır. Hoca polinom zamanda sonlanmayı önemli bir şart olarak anlatır. (Ders sonrası not: tanım büyük olasılıkla yalnız sonlu sürede bitmeyi ister; hocanın vurgusu pratikte makul sürede bitmektir.)
  - Doğruluk: algoritmanın verilen girdiler için beklenen çıktıyı ürettiği farklı durumlar (case) üzerinden kontrol edilmelidir.
- Algoritmayı ifade etme yolları (ekrandan, 02:05:20)
  - Programlama dilleri (C, Python, C++, Java…).
  - Akış diyagramı (flowchart notation).
  - Sözde kod (pseudocode): kod ile konuşma dili arasında, kodun sadeleştirilmiş hâli.
  - Sözel anlatım (oral presentation).
  - Bu derste önce akış diyagramı, sonra C kullanılacaktır.
- Akış diyagramı sembolleri
  - START / STOP: oval. Her algoritmanın başlangıcı ve bitişi vardır.
  - Giriş (input, girdi): değişkene dışarıdan değer okunur. Örnek: `a`.
  - Çıkış (output, çıktı): kullanıcıya sonuç gösterilir. Örnek: `b`. Sabit mesajlar tek tırnak içinde yazılır ve değişkenlerden virgülle ayrılır: `'Sonuç=', result`.
  - İşlem (process): dikdörtgen. Atama ve hesaplamalar yapılır (`x = a + b`). Bir kutuda birden fazla işlem yazılabilir, ama genellikle 4-5’i geçmemesi istenir.
  - Karar (if): koşul kontrol edilir. Doğru (T) ve yanlış (F) olmak üzere iki kolu vardır.
  - Bağlantılar oklarla gösterilir. Sayfa veya yer biterse numaralı bağlantı daireleri (1, 2…) kullanılır.
  - Akış diyagramlarında değişkenler tanımlanmadan kullanılabilir. Programlama dillerinin çoğunda ise tanımlanmaları gerekir.
- Kullanılacak işlemler
  - Aritmetik: `+`, `-`, `*`, `/`, `mod` (kalan), `div` (tam sayı bölmesi, ondalık kısım atılır, yuvarlama yapılmaz).
  - Karşılaştırma: `>`, `<`, `>=`, `<=`, `=`, ≠.
  - Atama: `B = A`, `C = 5`, `A = A + B` (A’nın mevcut değeri B ile toplanıp yine A’ya yazılır).
  - = işareti işlem kutusunda atama, karar kutusunda karşılaştırma anlamına gelir. C dilinde karşılaştırma için ==, atama için = kullanılır. Akış diyagramında karşılaştırma için == yazmak da yanlış sayılmaz.
- Örnek 1: İki sayıyı toplayan algoritma
  - START → çıkış: `'İki sayı giriniz'` → giriş: `A, B` → işlem: `C = A + B` → çıkış: `'Sonuç=', C` → STOP.
  - Kullanıcıya ne girmesi gerektiği önce bir mesajla bildirilmelidir.
- Örnek 2: Dört işlem yapan algoritma (ekrandan, 02:29:40)
  - START → `'İki sayı giriniz'` → giriş: `number1, number2` → `'İşlem tipini giriniz'` → giriş: `operator`.
  - Art arda karar kutuları: `operator = '+'` ise `result = number1 + number2`, değilse `operator = '-'` kontrolü, değilse `'*'`, değilse `'/'`. Hiçbiri değilse `'Illegal Operator'` mesajı verilir.
  - Bir karar kutusu aynı anda dört durumu kontrol edemez. Her işlem için ayrı bir karar kutusu gerekir.
  - Karakter sabitleri (`'+'`) tek tırnak içinde yazılır.
  - Kollar birleştirildikten sonra `'Sonuç=', result` yazdırılır ve STOP.

Hocanın Özellikle Vurguladığı Kısımlar

- Sıkıştırma yöntemi veriye göre seçilmeli
  - RLE tekrarı az olan veride veriyi büyütür. Algoritma seçilirken verinin yapısına bakılmalıdır.

- Parity bit yalnızca tek bitlik hataları güvenle yakalar
  - Çift sayıda bit değişirse hata fark edilmez.
- Algoritmada belirsizlik olmamalı
  - Her adım tek anlamlı ve kesin olmalıdır. Algoritma mutlaka sonlanmalı ve doğruluğu farklı girdilerle kontrol edilmelidir.
- Önce akış diyagramı, sonra kod
  - Bu derste bir soruda yalnızca kod yazılırsa, aksi belirtilmedikçe değerlendirme dışı bırakılır. Önce akış diyagramı çizilmelidir.
- If yapısı kuralı
  - True kolu olmayan, yalnızca false kolu olan bir if tasarlanmamalıdır. Böyle bir durumda koşul ters çevrilmelidir. False kolu ise boş bırakılabilir.
- Sonucu bir değişkende saklayın
  - Öğrenme aşamasında hesaplama sonucu doğrudan çıkışa yazılmak yerine bir değişkende (ör. `result`) saklanıp sonra yazdırılmalıdır.
- İlk hedef doğru çalışan algoritma
  - Dönemin yarısına kadar hedef, verilen girdi için istenen çıktıyı üreten doğru bir algoritmadır. Verimlilik ikinci hedeftir.
- Düzgün bir defter tutun
  - Ekran görüntüsü alınsa bile notların temize çekilerek bir defterde toplanması tavsiye edilir.

Kısa Tekrar Notları

- Sıkıştırma = encoding + decoding. Lossless: birebir geri döner. Lossy: kayıp vardır.
- RLE: `AAABBCCCDDEEF` → `3A2B3C2D2E1F`. Tekrar azsa veriyi büyütür.
- Frekansa bağlı kodlama: sık sembol → az bit (Huffman).
- Relative encoding: videoda yalnızca kareler arası fark gönderilir.
- Dictionary encoding: kelimeler küçük bloklardan üretilir.
- Parity bit: odd/even. Tek bitlik hatayı yakalar, çift bitlik hatayı yakalayamaz.
- Hamming uzaklığı = farklı bit sayısı. Kodlar arası uzaklık ≥ 3 ise tek bitlik hata düzeltilebilir.
- Algoritma: sıralı, belirsizlik içermeyen, uygulanabilir, sonlanan adımlar.
- Gösterimler: programlama dili, akış diyagramı, pseudocode, sözel anlatım.
- Akış diyagramı: START/STOP (oval), input, output, process (dikdörtgen), if (karar, T/F kolları).
- `div` = tam sayı bölmesi, `mod` = kalan.
- = işlem kutusunda atama, if içinde karşılaştırmadır (C'de ==).

Detaylı Açıklamalar

Dersin ilk bölümü veri sıkıştırmayla başlar. Sıkıştırma, verinin bir algoritmayla daha az yer kaplayacak biçimde kodlanması ve kullanılacağı zaman tekrar çözülmesidir. Çözme sonunda orijinal verinin birebir aynısı elde ediliyorsa sıkıştırma kayıpsız, elde edilemiyorsa kayıplıdır. Her dosyanın her zaman sıkıştırılmamasının nedeni, kodlama ve çözme işlemlerinin işlemciye ek yük getirmesidir. Sıkıştırılmış veri doğrudan kullanılamaz ve her erişimde önce çözülmesi gerekir. Aynı kodlama/çözme mantığı veri güvenliğinde şifreleme ve şifre çözme olarak da karşımıza çıkar. Ancak iki işlemin amacı farklıdır: sıkıştırma veriyi küçültmeye, şifreleme ise veriyi yetkisiz kişilere anlaşılmaz kılmaya çalışır.

Run-Length Encoding (RLE), art arda tekrar eden değerleri "tekrar sayısı + değer" biçiminde saklar. Tahtadaki piksel örneğinde 12 byte'lık `10 10 10 10 255 255 255 0 0 0 0 5` dizisi `4 10 3 255 4 0 1 5` olarak 8 byte'a iner. Bu yaklaşık %33'lük bir kazançtır. Aynı yöntem `AAABBCCCDDEEF` metninde 13 byte'ı ancak 12 byte'a indirebilir. Tekrarı olmayan `ABCDEF` gibi bir veride ise veri 6 byte'tan 12 byte'a çıkar. Bu yüzden RLE, gökyüzü veya deniz gibi geniş tek renkli alanlar içeren görüntülerde verimlidir. Frekansa bağlı kodlamada sık kullanılan semboller kısa, seyrek kullanılanlar uzun bit dizileriyle kodlanır. Böylece metnin toplam boyutu azalır. Huffman kodlaması bunun bilinen bir örneğidir. Göreli kodlamada ise art arda gelen video kareleri arasındaki fark hesaplanır ve yalnızca değişen kısım saklanır ya da gönderilir. Sözlük kodlaması, kelimeleri daha küçük ortak bloklardan oluşturarak tekrar eden parçaları bir kez saklar.

Sıkıştırmanın faydaları yalnızca depolama alanıyla sınırlı değildir. Ağ üzerinden veri gönderilirken bant genişliği sınırlıdır ve büyük veriler gecikmelere yol açar. Daha küçük veri hem daha hızlı iletilir hem de mobil kotalardaki maliyeti azaltır. Ağ üzerindeki veriler günümüzde düz metin olarak değil, şifrelenerek gönderilir. Günlük hayatta RAR, ZIP ve 7z genel arşivleme, MP3 ve FLAC ses, JPEG ve GIF resim, MPEG ve MKV video için kullanılan sıkıştırma formatlarıdır. Ayrıca bir görüntüdeki yazıyı metne çeviren OCR yöntemi de kısaca anlatılır.

Dersin ikinci bölümü iletişim hatalarıdır. Bir birimden diğerine ya da bir bilgisayardan diğerine aktarılan verideki bir bitin değişmesi bile verinin anlamını bozar. Bu hataları tespit etmenin en basit yolu eşlik bitidir. Veriye bir bit daha eklenir ve bu bit, toplam 1 sayısını tek (odd parity) ya da çift (even parity) yapacak şekilde seçilir. Örneğin tek eşlikte `01000001` verisine 1, `01000110` verisine 0 eklenir. Alıcı 1'lerin sayısını kontrol eder ve beklenen durum bozulmuşsa hata olduğunu anlar. Ancak iki bit aynı anda değişirse teklik/çiftlik korunur ve hata fark edilmez. Bu yüzden eşlik biti tek bitlik hatalar için uygun basit bir yöntemdir.

Hatanın düzeltilmesi için Hamming uzaklığından yararlanılır. Hamming uzaklığı iki bit deseni arasındaki farklı bit sayısıdır. Tahtadaki örnekte 8 sembol normalde 3 bitle ifade edilebilecekken, her iki kod arasındaki uzaklık en az 3 olacak şekilde 6 bitlik kodlar seçilmiştir. Alıcıya `010100` gibi tabloda bulunmayan bir desen geldiğinde, bu desenin her koda uzaklığı hesaplanır. En küçük uzaklık D koduna (`011100`) aittir ve değeri 1'dir. Desen D olarak düzeltilir. Kodlar arasındaki uzaklık en az 3 olduğu için tek bitlik bir hata her zaman tek bir koda en yakın kalır. Bunun bedeli, fazladan bit kullanmaktır. Hata tespiti ile verinin yeniden istenmesi ya da fazla bitle hatanın yerinde düzeltilmesi arasında ihtiyaca göre seçim yapılır. Disklerdeki RAID düzenlerinde de alanın bir kısmı bu tür kurtarma bilgisine ayrılır.

Son bölümde algoritma kavramına giriş yapılır. Algoritma; sıralı, belirsizlik içermeyen, uygulanabilir ve sonlanan adımlar kümesidir. Bu tanımdaki her özellik önemlidir. Adımlar tek anlamlı olmalıdır ("100 ml su dök" gibi). Algoritma bir başlangıç ve bitişe sahip olmalı, makul sürede sonuçlanmalı ve farklı girdiler için doğru çıktıyı ürettiği kontrol edilmelidir. Algoritmalar programlama dilleri, akış diyagramları, sözde kod ya da sözel anlatımla ifade edilebilir. Bu derste önce akış diyagramı, ardından C dili kullanılır. Akış diyagramında START ve STOP ovalleri, giriş ve çıkış kutuları, işlem dikdörtgenleri ve iki kollu karar kutuları kullanılır. İşlemler için aritmetik operatörler, `mod`, `div`, karşılaştırma operatörleri ve atama kullanılır.

İki örnek çizilir. İki sayıyı toplayan algoritmada kullanıcıya önce bir mesaj gösterilir, iki sayı okunur, toplam bir değişkene atanır ve sonuç bir mesajla birlikte yazdırılır. Dört işlem yapan algoritmada ise iki sayı ve işlem tipi okunduktan sonra işlem karakteri art arda karar kutularıyla `'+'`, `'-'`, `'*'` ve `'/'` sabitleriyle karşılaştırılır. Eşleşen koldaki işlem yapılır, hiçbiri eşleşmezse "Illegal Operator" mesajı verilir. Tüm kollar birleştirilerek sonuç yazdırılır.

Ders sonunda bir sonraki ders için bir problem verilir. Problem, üç basamaklı bir sayının (örneğin 125, 722, 893) basamaklarının küpleri toplamının sayının kendisine eşit olup olmadığını bulan algoritmanın akış diyagramını çizmektir. Sayı tek bir sayı olarak okunur, rakam rakam verilmez. Yalnızca dört işlem, `mod`, `div` ve karşılaştırma operatörleri kullanılır. Hoca bu sayılara "kübik sayı" der. Bir öğrenci bu tür sayıların literatürde Armstrong sayısı olarak geçtiğini söyler. Hoca bunu kontrol edip bir sonraki derste bilgi vereceğini belirtir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.